

Voltage & Ripple Module

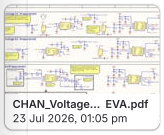
- Conext
- Requirements
- Main design decision: split DC and ripple paths
  - DC voltage measurement path
    - ADC LSB equations
    - DC ranges
    - High-range calculation
  - Ripple measurement path
    - Why gain is needed
    - Ripple frontend example values
  - ADC and part-selection
  - MCP33131 Justification using measured ripple case
  - Op Amp Selection
    - Op-amp bandwidth calculation
    - Op-amp slew rate calculation
    - Op-amp offset error calculation
  - AC coupling capacitor calculation
  - Ripple sensitivity calculations
    - Ripple gain stage
    - Low & High ripple path
    - Ripple headroom calculation
  - Anti-alias filter calculation
- STM32 TINGS
  - STM32 Basic Throughput check
  - STM32 Crystal Justification
    - Crystal Oscillator Choice
    - Summary of Results:
  - STM32 Memory for Samples
  - STM32 Automated FFT Approaches
    - Investigating Local FFT (Real FFT)
      - Using int16_t Real FFT
      - What local FFT can realistically do
    - Investigating Motherboard FFT @Ricky Sun
      - Data rate calculation
    - Motherboard STM32 requirements
      - STM32 motherboard MCU candidates
- Protection
  - Clamp current check at accidental 36 V
  - ADC Protection at accidental 36 V
- Firmware Equations & Calibration
- Spice Simulations

This document contains my ALL design design decisions and ALL major justifications/calculations for C.H.A.N Voltage & Ripple Daughter Module REVA.

To make life easier:
Parent “Heading 1’s” have a summary of findings for that entire section

Child Headings have lower level TLDR for concise summary

Schematic REVs	Notes



RevA
(Shared to ANT61 Altium aswell)

15 Jul 2026

- Pending System Architecture review
- Pending Schematic Review

My Concerns:

- Accidental 36V input protection may not be robust enough.
- STM32 Automated FFT Section, sanity check my maths and understanding pls. Some knowledge in this section is based of youtube & chatgpt.
- Overall please roast me, feel free to scrutinize anything and everything

Open Question:

- Protection - May i assume we wont be inputing >5Vpp AC into this. AC path has minimal protection.

Spice Checklist (Green = Completed)

Area	What to check	Goal	Method	Method
DC divider paths	Low/high range scaling	ADC input never exceeds 4.096 V in valid range	Hand calc	SPICE
Protection clamps	36 V accidentally on low range	Clamp current is safe	Hand calc	SPICE
Ripple path	AC coupling, gain, VCM bias	10 mV ripple is measurable	SPICE Bode + transient	
Anti-alias filter	30 kHz passband, high-frequency rolloff	30 kHz not overly attenuated	Hand calc	SPICE AC sweep
Op-amp output swing	Worst-case ripple plus DC bias	Op-amp does not clip	Hand calc	SPICE transient
SPI bandwidth	ADC SPI plus motherboard SPI	Data rate is feasible	Hand calc	
AC couple	Confirm high-pass cutoff 15.9 Hz	High-pass is high passing	Hand Calc	SPICE AC sweep

Ripple gain stage	Transient sim with 10 mV sine input	Confirm expected amplified ripple amplitude	Hand Calc	SPICE AC sweep
Ripple gain linearity	Sweep 10 mV, 20 mV, 50 mV, 100 mV ripple	Confirm output scales linearly and does not clip	SPICE AC sweep	
VCM bias stability	Transient/start up sim of VCM buffer	Confirm VCM settles to 2.048 V cleanly	SPICE	

Conext

The tester motherboard provides power, SPI communication, and a crossbar matrix. The crossbar matrix selects which Beacon test point is routed to the daughter board. The daughter board should not assume it knows which rail is connected; the motherboard firmware should tell it which measurement range/path to use.

Beacon PCB test pads → Motherboard crossbar matrix → Voltage + Ripple daughter board
Local analog frontend + ADC + STM32 → SPI response to motherboard: board ID, DC voltage, ripple result, status

Requirements

Requirement	Interpretation	Design
Measure up to 36 V	Any selected CrossBar node may be as high as 36 V.	Use a 36 V-safe divider and protection before ADC/op-amp stages.
Possibly measure ripple on 36 V node	Ripple may sit on top of a large DC offset.	Use a separate AC-coupled ripple path after a safe divider.
Measure ripple around 10 mV	resolution target	Add gain after AC coupling. Raw 0-36 V ADC range is too coarse.
Ripple frequency at least 30 kHz	The analog path must pass 30 kHz. For waveform amplitude, sample rate should be higher than Nyquist minimum.	Use op-amp/ADC capable of at least 150-300 kSPS for useful amplitude measurement
ADC at least 12-bit, preferably 16-bit	16-bit is better for margin.	MCP33131-class is better for final ripple capture.
SPI interface and unique board ID	Motherboard must identify the daughter	Use STM32 as the SPI slave to motherboard

	board and receive results.	
Powered from motherboard 3.3 V / 5 V	No external supply should be required.	
Current below 600 mA	Do not overload motherboard power.	Analog frontend should be well below this
FFT Plot	Good Engineering, make goated hardware. Use 100% brain capacity.	Current bottle neck is STM32F1 SPI. 18Mbit/s with each ADC sample taking 24 SPI clocks yields a maximum sample of: 750kSPS. Current reason for STM32F1 is BOM consolidation with other CHAN modules. Potentially add a precise HSI for STM32.

Main design decision: split DC and ripple paths

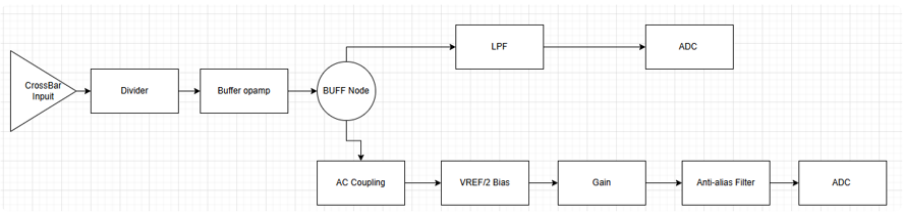
TLDR

Voltage on nodes are split into DC and AC components, due the high range of possible inputs (0 - 36V) the voltage dividers “attenuate“ the AC component too much.

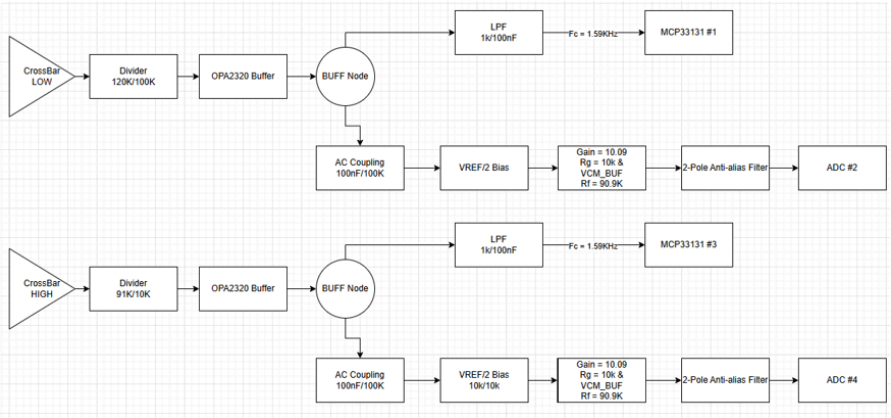
Summary:

- DC Voltage Measurement Paths - Need a 16 Bit ADC
- Ripple measurement Path - Need Gain because of voltage division
- MCP33131 is a good choice for both measured and targeted ripple
- Op Amp is within spec
- Plenty Ripple headroom
- Automated FFT Plot is Feasible
- Current Sampling Rate limitation is the STM32F1 at 750kSPS. It is NOT an issue.

A single 0-36 V ADC path is not enough for good ripple measurement because the same divider that protects the ADC also attenuates the ripple. For example, a 10 mV ripple through a 0.099 divider becomes only 0.99 mV at the ADC input before gain. Therefore, the module should have two measurement paths:



High level Diagram



Final Schematics Plan, Note - each path has its own buffer (4 buffers)

High Level Overview:

- 2 Inputs, 0-8V & 0-36V
  - Difference between each is the voltage divider ratio
- Buffer 1 for Low impedance output
- DC measurement
  - LPF → ADC
- Ripple Measurement
  - AC couple → Bias to Vref/2 → Gain → Anti-Alias Filter → ADC

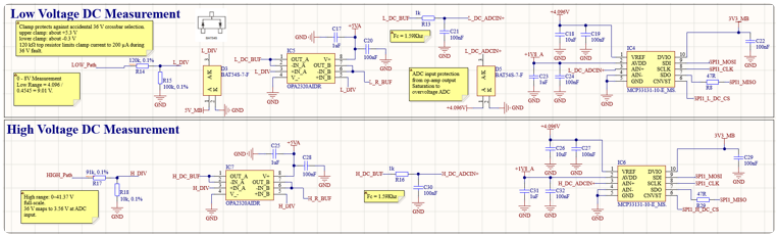
DC voltage measurement path

TLDR

Calculated the smallest detectable change using 12- and 16-bit ADCs. The 12-bit ADC lacks sufficient resolution for a 0-36V range. Chose the 16-bit ADC.

Also proved why AC coupling path is needed to meet Ripple measurements in the order of mV.

CrossBar input → input protection → voltage divider → buffer op-amp → low-pass filter → ADC → STM32



ADC LSB equations

For an N-bit ADC with reference voltage VREF:

ADC-input LSB:
V_LSB_ADC = V_REF / 2^N

Node-side LSB after a divider:
V_LSB_node = V_REF / (2^N * k)

Divider ratio:
k = R_bottom / (R_top + R_bottom)

For MCP3201 with VREF = 4.096 V and N = 12:

2^12 = 4096
V_LSB_ADC = 4.096 V / 4096 = 0.001 V = 1 mV/count

DC ranges

Range	Divider example	k	Node full-scale with 4.096 V ref	Node LSB with 12-bit MCP3201	Node LSB with 16-bit MCP33131	Use case
Low range	120 k top, 100 k bottom	0.4545	9.01 V	2.20 mV/count	138 uV/count	3.3 V, 5 V, battery, 8 V rails with headroom.
High range	91 k top, 10 k bottom	0.0990	41.37 V	10.10 mV/count	631 uV/count	36 V-capable range.

High-range calculation

Chose R_top = 91 k and R_bottom = 10 k.

k = R_bottom / (R_top + R_bottom)
k = 10 k / (91 k + 10 k)
k = 10 / 101
k = 0.0990

At 36 V:
V_ADC = V_node * k
V_ADC = 36 V * 0.0990
V_ADC = 3.56 V

Full-scale node voltage:
V_FS_node = V_REF / k
V_FS_node = 4.096 V / 0.0990
V_FS_node = 41.37 V

Node-side LSB:
V_LSB_node = 41.37 V / 4096
V_LSB_node = 10.10 mV/count

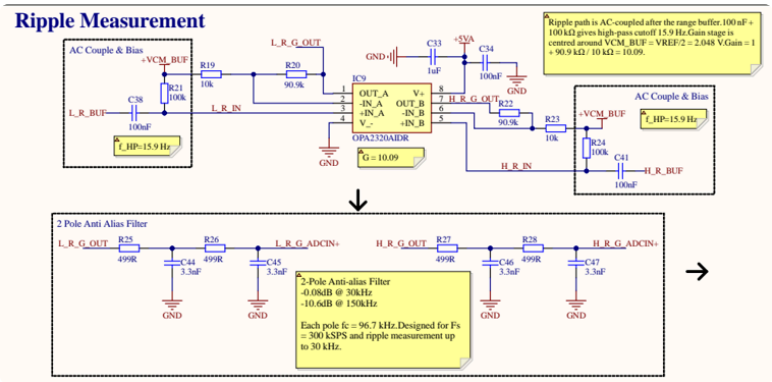
what it means: the 36 V range is safe, but a 10 mV change at the node is approximately one ADC count. That is acceptable for coarse DC checks but **too marginal for accurate ripple measurement** without a separate gain stage or a higher-resolution ADC.

Ripple measurement path

TLDR

Gain is necessary because the voltage divider reduces the ripple magnitude. A 10mV ripple target after the divider is smaller than the LSB size for 12-bit resolution. It is adequate for 16-bit resolution, but an accurate FFT plot requires more points.

Preliminary Calculations for AC coupling, Biasing, filtering.



The ripple path should remove the DC component and amplify the small AC component before sampling.

CrossBar input → input protection → 36 V divider → AC coupling capacitor → VREF/2 bias → gain amp → anti-alias filter → ADC → STM32

Why gain is needed

Using high-range divider k = 0.0990:

For 10 mV ripple at the Beacon node:
V_ripple_after_divider = 10 mV * 0.0990 = 0.99 mV

MCP3201 with 4.096 V reference has 1 mV/count at the ADC input.
So 0.99 mV is about 1 count before gain. This is too close to quantisation and noise.

With gain = 20:
V_ripple_after_gain = 10 mV * 0.0990 * 20 = 19.8 mV
ADC counts = 19.8 mV / 1 mV/count = 19.8 counts

Ripple case	Node ripple	After 0.099 divider	After gain = 20	MCP3201 counts
Minimum useful target	10 mV	0.99 mV	19.8 mV	19.8 counts
Measured worst case Ripple Measurements	87.2 mV	8.63 mV	172.7 mV	172.7 counts
Larger margin case	200 mV	19.8 mV	396 mV	396 counts

Ripple frontend example values

Function	Example implementation	Notes / calculation
High-voltage divider	91 k top, 10 k bottom, 0.1% preferred	36 V becomes 3.56 V.
AC coupling	Place capacitor after divider	Preferred because the capacitor does not need full 36 V DC input.

Bias point	VREF/2 = 2.048 V	maximise AC waveform swing
High-pass filter	100 k and 100 nF	fc = 1 /(2PiRC) = 15.9 Hz. This is well below the measured 420 Hz fundamental. Ripple Measurements
Gain	Non-inverting gain around 20	G = 1 + Rf/Rg. Rg = 10 k, Rf = 191 k gives G = 20.1.
Anti-alias low-pass	1 k + 10 nF	fc = 15.9 kHz. measuring known 2.52 kHz ripple.
ADC sampling	Prefer 150-300 kSPS minimum for 30 kHz amplitude capture	60 kSPS is Nyquist minimum

ADC and part-selection

Part / choice	Use	Pros	Limitations / notes
MCP3201	Prototype ADC for DC and measured low-frequency ripple	12-bit, SPI, simple, 100 kSPS class part.	Single channel and marginal for 10 mV ripple on 36 V range. Not enough margin for arbitrary 30 kHz amplitude measurement.
MCP33131D-10 / MCP33131-class ADC	Good ADC for ripple waveform capture	16-bit, single channel, 1 MSPS class ADC.	Differential high-speed SAR ADC; not a drop-in replacement. It needs a suitable input driver/reference/layout.

the motherboard SPI talks to the daughter-board STM32. The STM32 identifies the board, controls the local ADCs, stores calibration constants, performs averaging or FFT, and returns processed results.

MCP33131 Justification using measured ripple case

TLDR

2.52kHz 5th harmonic is the current documented ripple. However from previous QA, 30Khz was the decided targeted.
To get 10 samples/cycle at 30Khz, 300kSPS is needed. Thus the 12-Bit **MCP3201** at 100kSPS is not sufficient, and the 16-Bit **MCP33131** is justified with 500KSPS - 1MSPS.

> Adding sufficient headroom in sampling rate both future proofs the module and increases the accuracy of the automated FFT plot.

Ripple Measurements

The known worst-case measurement was on the 5 V line using two batteries: 87.2 mV ripple, fundamental around 420 Hz, with listed components up to the 5th harmonic at 2.52 kHz.

Component	Frequency	Level from notes
Fundamental	420 Hz	-39.40 dBV
Peak between	680 Hz	-47 dBV
1st harmonic	840 Hz	-47.60 dBV
Peak between	1.104 kHz	-48 dBV
2nd harmonic	1.26 kHz	-60.0 dBV
Peak between	1.524 kHz	-49.0 dBV
3rd harmonic	1.68 kHz	-63.80 dBV
Peak between	1.940 kHz	-51.80 dBV
4th harmonic	2.1 kHz	-58 dBV
Peak between	2.37 kHz	-57.20 dBV
5th harmonic	2.52 kHz	-58 dBV

For known 2.52 kHz highest listed component:
Samples per cycle at 100 kSPS = 100000 / 2520 = 39.7 samples/cycle
This is enough for detection and approximate amplitude measurement.

For general 30 kHz ripple:
Nyquist minimum = 2 * 30 kHz = 60 kSPS
Samples per cycle at 100 kSPS = 100000 / 30000 = 3.33 samples/cycle
This is not enough for robust amplitude measurement.

Practical target for amplitude:
5 samples/cycle at 30 kHz = 150 kSPS
10 samples/cycle at 30 kHz = 300 kSPS
MCP33131-10-E/MS | DigiKey Electronics (16 bits) - 500KSPS - 1MSPS

MCP3201 (12 bits) - 100KSPS

Op Amp Selection

> TLDR
> Verified Bandwidth with the required 30Khz & also will not make ADC's Max Sample rate redundant (both around 1Mhz)
>
> Verified Slew Rate with plenty of headroom 10V/us >> 0.339V/us.
>
> Verified offset error with a maximum of 1.52mV on the High range measurement path.

OPA2320AIDR | DigiKey Electronics - **three dual OPA2320 packages**.

Using this op amp for:

- Low-range buffer

- High-range buffer
- Low-ripple gain stage
- High-ripple gain stage
- VREF/2 bias buffer

Op-amp bandwidth calculation

Ripple requirement:

$$f_{ripple_max} = 30kHz$$

Ripple gain target:

$$G = 10$$

For a simple check, closed-loop bandwidth is approximately:

$$BW_{closedloop} = GBW/G$$

OPA2320 gain-bandwidth:

$$GBW=20MHz$$

With gain 20.1:

$$BW_{closedloop} = 20MHz/20.1$$

Compare with 30 kHz:

$$0.99MHz >> 30kHz$$

So bandwidth is fine.

Op-amp slew rate calculation

Worst useful ADC swing is roughly:

$$V_{swing} = 1.8V$$

because the ripple output is biased at 2.048 V and must stay inside 0 to 4.096 V.

Required slew rate for a sine wave is:

$$SR_{req} = 2*Pi*fV_{swing}$$

At 30 kHz:

$$SR_{req} = 2*Pi*(30k)(1.8)$$

$$SR_{req} = 0.339V/us$$

OPA2320 slew rate is about:

$$SR_{OPA} = 10V/us$$

So:

$$10V/us >> 0.339V/us$$

Slew rate is also fine.

Op-amp offset error calculation

OPA2320 max offset is about:

$$V_{OS} = 150uV$$

For the low DC path, node-equivalent error is:

$$E_{OS,L} = V_{OS}/voltage_divider_ratio = 150uV/0.4545$$

$$V_{EOS,L} = 330uV$$

For the high DC path:

$$E_{OS,L} = V_{OS}/voltage_divider_ratio = 150uV/0.0990$$

$$V_{EOS,L} = 1.52mV$$

These are small.

AC coupling capacitor calculation

> TLDR
>
> High Pass Cutoff at 15.9Hz.
> Well margined to the measured 420 Hz ripple fundamental and far below the 30 kHz requirement

This removes the DC voltage and leaves only ripple centered around 2.048 V.

High-pass cutoff:

$f_{HP} = 1/2*Pi*R_{BIAS}*C_{AC}$

C_{AC} = 100 nF, X7R, 25 V

R_{BIAS} = 100 kohm, 0.1%

f_{HP} = 15.9Hz - far below known 420 Hz ripple fundamental and far below the 30 kHz requirement, so the ripple of interest passes through.

Ripple sensitivity calculations

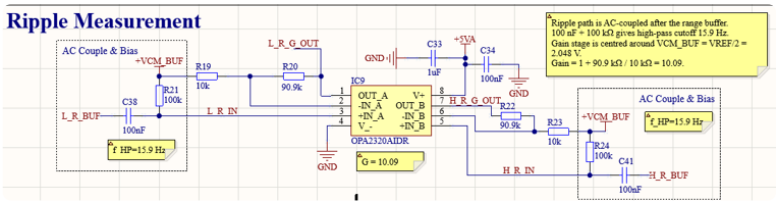
> TLDR
>
> - Non Inverting Gain = 10
> - Verified a 10mV ripple results in a Minimum of 160 ADC counts on High Range measurement
> - After Biasing to the center of the ADC reference voltage (4.096V/2), calculated the max ripple before clipping at the input to be 0.786 Vpp
>   - You would need some serious **wobble wobble** to exceed this

Ripple gain stage

non-inverting gain around VCM_BUF

Rf = 90.9k & Rg = 10k, G = 10.09\

Low & High ripple path



For 10 mV ripple at the Beacon node:

$V_{ADC,ripple,L} = 10mV * divider_ratio * G$
$V_{ADC,ripple,L} = 10mV * 0.4545 * 10.09$
$V_{ADC,ripple,L} = 45.9mV$

ADC counts:

$Counts_L = V_{ADC,ripple,L} / V_{LSB,ADC} = 45.9mV/62.5uV = 734$ counts

So 10 mV ripple on a low-voltage rail is easy to detect.

For a High Ripple Path

V_ADC,ripple,L = 10mV * divider_ratio * G
V_ADC,ripple,L = 10mV ∗ 0.099 ∗ 10.09
V_ADC,ripple,L = 9.99mV

ADC counts:

Counts_L = V_ADC,ripple,L/ V_LSB,ADC = 9.99mV/62.5uV = 160 counts

Still usable, on a 36 V node.

Ripple headroom calculation

The ripple output is centred at:

$$VCM=2.048V$$

The ADC range is:

$$0V \rightarrow 4.096V$$

Allow about:

$$V_{OUT,max} = 1.8V$$

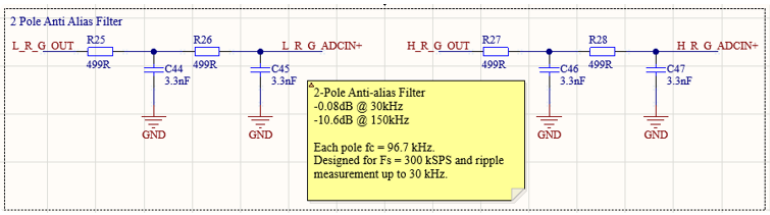
to avoid clipping.

Maximum Ripple is:
V_ripple,node,max = 1.8/divider_ratio*G
gain of about 10 is a good compromise:

Path	10 mV ripple sensitivity	Approx max ripple before clipping
Low ripple	734 counts	0.786 Vpp
High ripple	160 counts	3.60 Vpp

Anti-alias filter calculation

> TLDR
>
> - Low pass filter with a steeper roll off to attenuate frequencies >= Nyquist.
>   - This is needed due to the GBP of the opamp pass band > Minimum Sampling rate.
>   - 16-Bit ADC can do 500kSPS minimum (sampling faster is better for atten aswell)
>   - I added this for due diligence, and 2 extra passives do not significantly impact cost or board space. The values are flexible if poles need to move.
> - Why 2 poles? better HF suppression. Suppressing Alias is important for a quality Automated FFT Plot



We want ripple up to 30Khz from the spec.
Thus we sample around f_s = 500kSPS which gives 500kSPS/30kHz = 17 samples per cycle
Nyquist freqeucny = 250kHz
So the filter should:

1. Let 0–30 kHz target pass reasonably well.
2. Start reducing frequencies above that.
3. Strongly reduce frequencies near/above the ADC Nyquist frequency, 250Khz

This is to prevent frequencies above nyquist from folding down to lower frequency.
I decided to go for a 2 pole filter to retain the frequency content above specification, with a sharper roll off before a sharper cuttoff at nyquist.

Filter	Attenuation
1-pole RC	-8.8 dB
2-pole RC	-17.7dB

Using a 2 pole RC filter after each ripple amplifier:
499ohms → 3.3nF to AGND → 499ohms → (Analogue IN+) → 3.3nF to AGND
Each pole has cuttoff, f_c = 96.7kHz

Attenuation at @ 30kHz:
A1 = 1/(sqrt(1+(f/f_c)^2)) = 0.955
and thus at 2 poles: A2 = A1^2 = 0.912 = -0.8dB.
30kHz is attenuated by 0.83dB which is acceptable, can also be corrected in firmware i think.

Attenuation @ 150kHz:
A1 = 1/(sqrt(1+(f/f_c)^2)) = 0.542
and thus at 2 poles: A2 = A1^2 = 0.294 = -10.6dB.
So filter starts suppressing freq strongly content near 300kSPS Nyquist.

Attenuation @ 250kHz:
A1 = 1/(sqrt(1+(f/f_c)^2)) = 0.361
and thus at 2 poles: A2 = A1^2 = 0.130 = -17.7dB.
So filter starts suppressing freq strongly content near 500kSPS Nyquist.

Measured worst Case Lower Frequency Ripple @ 2.52khz:

A1 = 1/(sqrt(1+(f/f_c)^2)) = 0.99966
and thus at 2 poles: A2 = A1^2 = 0.9993 = -0.006dB.
So for the known measured ripple case, the filter has basically no meaningful attenuation.

Note: Use C0G/NP0 caps.

STM32 TINGS

Summary:

- Sampling is bottlenecked by STM32F1 SPI, it is not an issue
- Selected External Crystal to ensure Quality FFT plot

STM32 Basic Throughput check

Sampling is bottlenecked by STM32F1 SPI

- Maximum Sample @1 Channel only = 750kSPS
- Maximum Sample @2 Channel simultaneous = 300kSPS
- The bottleneck is not a concern.

The MCP33131 family can run up to 1 Msps and can do faster SPI than the STM32F103, so the STM32 SPI is the bottleneck.

The MCP33131-10 data sheet shows **1 Msps** and **60 MHz SCLK**, while STM32F103 SPI is around **18 Mbit/s max**.

Assume each ADC sample needs about **24 SPI clocks** after overhead.

For one ADC:

$$f_{sample,max}=18MHz/24$$

$$f_{sample,max}=750kSPS$$

For two ripple ADCs sampled equally:

$$f_{sample,max}=750kSPS/2 = 375kSPS$$

The target for 30 kHz ripple was roughly 300 kSPS.

Thus two ripple channels at 300 kSPS each is possible.

this is also acceptable because the DC ADCs do not need high speed, can capture faster when ripple is needed.

STM32 Crystal Justification

> The Oscillator is well suited for FFT measurements and clock requirements.
>
> FFT Capturing for 0.1Sec @ 500kSPS yields 10Hz/bin.
>
> Measured Ripple at 30kHz:
>
> - The Internal HSI would yield +-300Hz error, which is quite large for a quality FFT plot.
> - The chosen External HSI would yield +-2.55Hz error, which is better suited.

For ripple frequency measurement or FFT plots, the sample rate accuracy matters. ST datasheet specifies a 1% Typ HSI Clock accuracy:
https://mm.digikey.com/Volume0/opasdata/d220001/medias/docus/1139/STM32F103x8%2CB.pdf

High-speed internal (HSI) RC oscillator

Table 24. HSI oscillator characteristics(1) (2)

Symbol	Parameter	Conditions	Min	Typ	Max	Unit
f_{HSI}	Frequency			8		MHz
ACC_{HSI}	Accuracy of HSI oscillator	T_A = –40 to 105 °C	–2	±1	2.5	%
		T_A = –10 to 85 °C	–1.5	±1	2.2	%
		T_A = 0 to 70 °C	–1.3	±1	2	%
		T_A = 25 °C	–1.1	±1	1.8	%
$t_{su(HSI)}$	HSI oscillator startup time		1		2	µs
$I_{DD(HSI)}$	HSI oscillator power consumption			80	100	µA

1. Guaranteed by design, not tested in production.
2. V_{DD} = 3.3 V, T_A = –40 to 105 °C unless otherwise specified.

E.g I am sampling at 300 kSPS,

If the clock is 1% slow, then the actual sample rate is not 300 kSPS.

It becomes:

$f_{s,actual} = 300,000 * (1 - 0.01)$

$f_{s,actual} = 300,000 * 0.99$

$f_{s,actual} = 297,000$ SPS

So we think we are sampling at 300,000 SPS but we are actually sampling at 297,000 SPS

When doing any FFT or frequency measurement, the frequency scale depends on the sample rate.

E.g the FFT frequency bins are calculated from: $f_{bin} = (bin_{num} * f_s) / Num_{of_Samples}$

So if f_s is wrong, all the frequency labels are wrong by the same percentage.

So a measured ripple at 30 kHz with 1% error could appear shifted by 30 kHz * 0.01 = 300 Hz on the FFT plot.

Crystal Oscillator Choice

I chose a 8MhZ HSE. Cext justified in schematics.

ECS-80-18-30B-AGN-TR | DigiKey Electronics

OPERATING CONDITIONS / ELECTRICAL CHARACTERISTICS

PARAMETERS	CONDITIONS	ECX-53B			UNITS
		MIN	TYP	MAX	
Frequency		8.000		50.000	MHz
Mode of Oscillation	Fundamental				
Frequency Tolerance*	@ +25°C			± 30	ppm
Frequency Stability*	-20 ~ +70°C			± 50	ppm
Shunt Capacitance	Co			5	pF
Load Capacitance	Specify in P/N	8	20	Series	pF
Drive Level	DL			100	µW
Operating Temperature*	Topr	-20		+70	°C
Storage Temperature	Tstg	-55		+125	°C
Aging (First Year)	@ +25°C ±3°C			±5	ppm

Using worst case error = freq tolerance + freq stability + 1st year aging = 30 + 50 +5 = 85ppm

f_error = 30kHz * 85/1000000 = 2.55Hz

Thus with this crystal a 30kHz ripple would be labeled as 30kHz +- 2.55Hz.

That is very small. Smaller is always better. 🥹

Summary of Results:

Clock source	Error assumption	30 kHz frequency error	Approx displayed range
STM32 HSI at 25°C	−1.1% / +1.8%	−330 Hz / +540 Hz	29.67–30.54 kHz
STM32 HSI over temp	−2% / +2.5%	−600 Hz / +750 Hz	29.40–30.75 kHz
ECS 8 MHz crystal	±85 ppm conservative	±2.55 Hz	29.980–30.020 kHz
ECS 8 MHz crystal tolerance only	±25 ppm	±0.75 Hz	29.999–30.001 kHz

STM32 Memory for Samples

Storing on the STM32F103C8 is feasible at:

- 2048 Samples, 2 channels
- 4096 samples, 1 channel

Will be streamed to Motherboard, thus this is the maximum capture window for 1 chunk.

STM32F103C8T7TR has 20 KB SRAM, keeping capture size sensible should be fine.

The MCP33131 gives 16 bit samples, thus 1 sample = 2 bytes (RAW ADC Capture)

$$\text{raw memory} = N\ (\#\ \text{samples}) \times 2\ \text{bytes} \times \text{number of channels}$$

$$2048\ \text{samples} \times 2\ \text{bytes} = 4096\ \text{bytes} = 4\ \text{KB}$$

Raw Capture is fine on the STM32F1, can even do 2 channels just fine. i.e Storing 2 ADC streams (low & high ripple)

Raw Capture Only

Capture case	Calculation	Memory
2048 samples, 1 channel	2048 × 2	4 KB
2048 samples, 2 channels	2048 × 2 × 2	8 KB
4096 samples, 1 channel	4096 × 2	8 KB
4096 samples, 2 channels	4096 × 2 × 2	16 KB

STM32 Automated FFT Approaches

> There are two FFT approaches:
>
> 1. Local FFT on the daughterboard STM32F103
> 2. External FFT on the motherboard STM32
>
> The local STM32F103 approach is simple and useful for basic measurements, but it is limited by the MCU's small SRAM. A 2048-point local int16_t real FFT is feasible and gives around 244 Hz/bin at 500 kSPS, but this is not high resolution.
>
> For high-quality FFT, the daughterboard should stream raw time-domain ADC samples to the motherboard. The motherboard reconstructs the longer capture window and performs one FFT over the full sample array.

Investigating Local FFT (Real FFT)

> The sample rate determines the maximum measurable frequency and the number of samples per ripple cycle. The capture time determines the FFT frequency resolution.
>
> The daughterboard **should not perform high-quality FFT locally** because it only has about **20 KB SRAM**. It is suitable for short captures, Vpp/RMS calculations, pass/fail checks, and possibly a small local FFT (resulting in 244 Hz/bin)

A local FFT means the STM32 processes the captured waveform into a frequency spectrum.

A normal floating-point complex FFT is memory heavy because each FFT point stores both real float and complex float.

Thus 1 complex float = 2*4 bytes = 8 bytes.

Thus a for "N" FFT Points: memory = N*8 bytes.

Yikes this takes alot of space.
With only 2048 point FFT, that would take up 16KB!

Local FFT on the STM32F103C8 can become tight.

Using int16_t Real FFT

Negative Frequency Half of FFT is redundant, so only use 1 half to conserve memory.

$$\text{FFT bins} = N/2 + 1$$

Also, because the STM32F103C8 does not have a hardware floating-point unit, int16_tfixed-point FFT is preferred over floating-point. int16_t = 2 bytes which matches the ADC sample size aswell.

Data type	Meaning	Bytes/sample	2048 samples	4096 samples
uint16_t raw ADC	ADC code from	2 bytes	4 KB	8 KB

	MCP33131			
int16_t	Mean-subtracted signed waveform	2 bytes	4 KB	8 KB
float waveform	Floating-point voltage/sample	4 bytes	8 KB	16 KB

What local FFT can realistically do

The FFT bin spacing is:

$$delta*F = f_s/N$$

where N = number of captured samples

At 500 kSPS, one channel:

Samples	Capture time	FFT bin spacing	Raw/Q15 memory	Local FFT feasibility
1024	2.05 ms	488 Hz/bin	2 KB	Feasible
2048	4.10 ms	244 Hz/bin	4 KB	Feasible
4096	8.19 ms	122 Hz/bin	8 KB	Tight
8192	16.38 ms	61 Hz/bin	16 KB	Not recommended
50,000	100 ms	10 Hz/bin	100 KB	Not possible

The STM32F1 Local FFT can realistically only do basic FFT, but not a high quality FFT.
I can still be useful to find an estimate for dominant frequency, rough harmonics and basic spectral pass/fail. Frequency spacing will not be fine, not high resolution and cant do a long capture window.

Investigating Motherboard FFT @Ricky Sun

> For high-quality FFT, the daughterboard should **stream sequential raw ADC sample chunks** to the motherboard. The motherboard then appends the chunks into one long time-domain capture and performs the FFT over the full reconstructed waveform.
>
> For high-quality FFT, the motherboard MCU needs enough RAM, CPU, and interface bandwidth.
>
> A 0.1 s capture at 500 kSPS gives:
>
> 50,000 samples/channel
> 10 Hz/bin FFT resolution
>
> But memory required is:
>
> 1 channel: 100 KB raw memory
> 2 channels: 200 KB raw memory
>
> (This is before FFT working memory & firmware etc.)
>
> Minimum sensible option: STM32H563 / H573 class, with better robust options listed below

Architecture:

- The STM32F1 module will
  - Read MCP33131 ADCs locally.
  - Sends chunks of raw time domain samples to the motherboard STM32
  - Motherboard appends chunks & runs one FFT over the full reconstructed array

Data rate calculation

2 channels at 500 kSPS (DMA Driven)

2 * 500,000 * 2 = 2,000,000 bytes/s = 2 MB/s = 16 Mbit/s raw

With overhead probably need 24 - 32 Mbit/S. (Exceeds STM32 F1 SPI limit of 18Mbit/s)

1 channels at 500 kSPS (DMA Driven)

1 * 500,000 * 2 = 1,000,000 bytes/s = 1 MB/s = 8 Mbit/s raw

With overhead probably need 12 - 16 Mbit/S. (Close to STM32 F1 SPI limit of 18Mbit/s)

streaming modes				
Mode	Data rate raw	target	Feasible with STM32F103 module SPI?	Use case
1 ch × 300 kSPS	4.8 Mbit/s	8 Mbit/s	Yes	Required Data rate
1 ch × 500 kSPS	8.0 Mbit/s	12–16 Mbit/s	Yes, tight	High-rate single channel
2 ch × 300 kSPS	9.6 Mbit/s	12–16 Mbit/s	Possible, tight	Dual ripple capture
2 ch × 500 kSPS	16.0 Mbit/s	24–32 Mbit/s	No	Needs faster interface

Motherboard STM32 requirements

For high-quality FFT, the motherboard needs enough RAM, CPU, interface bandwidth.

At 500 kSPS:

Capture mode	Capture time	Samples per channel	Raw memory needed	FFT resolution
1 ch	0.01 s	5,000	10 KB	100 Hz/bin
1 ch	0.1 s	50,000	100 KB	10 Hz/bin
2 ch	0.1 s	50,000/ch	200 KB	10 Hz/bin
2 ch	0.2 s	100,000/ch	400 KB	5 Hz/bin

Raw memory alone does not include FFT working memory, or firmware etc.

practical SRAM targets:

Motherboard target	Minimum SRAM	Better SRAM
Receive and forward raw data to PC/Externally	128 KB	256 KB
1-channel high-quality FFT locally	256 KB	512 KB
2-channel high-quality FFT locally	512 KB	1 MB

STM32 motherboard MCU candidates

MCU	CPU	SRAM	Why it fits / does not fit
STM32G474	Cortex-M4F, 170 MHz	128 KB	moderate buffering, but limited for 0.1 s two-channel FFT. 128 KB SRAM. (STMicroelectronics)
STM32F407/417	Cortex-M4F, 168 MHz	192 KB + 4 KB backup	Better than G4 for buffering, but still tight for two-channel high-quality FFT. 192 KB SRAM. (STMicroelectronics)
STM32H563/573	Cortex-M33 + FPU, 250 MHz	640 KB	Strong candidate. Enough RAM for two-channel 0.1 s raw capture and local FFT workspace. 640 KB SRAM. (STMicroelectronics)

STM32H743/753	Cortex-M7 + DP-FPU, 480 MHz	1 MB	Best STM32 option imo. Enough RAM and CPU for high-quality FFT, buffering, and future expansion. 1 MB SRAM and 480 MHz CPU. (STMicroelectronics)

Btw the STM32H743/753 is only 35 buckeroons. 😉🙏

Protection

It is possible to accidentally connect 36V to the LOW measurement path.
Op AMP & ADC protection have been considered and implemented,.

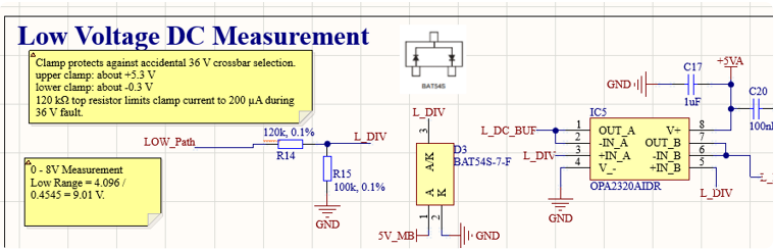
Concern - This shouldn't be considered a transient event, it is possible for significant time to pass before module realizes & disconnects.

Clamp current check at accidental 36 V

TLDR

- At full clamp, current it limited to 203uA.
- Does not exceed opamp rating

BAT54S-7-F | DigiKey Electronics



If L_DIV clamps to about 5.3 V:

I_TOP = (36 - 5.3) / 120k

I_TOP = 256 uA

Current through the 100 kohm bottom resistor:

I_BOT = 5.3 / 100k

I_BOT = 53 uA

So the current into the upper clamp diode is approximately:

I_CLAMP = 256 uA - 53 uA

I_CLAMP = 203 uA

That is small. The 120 kohm resistor limits the fault current, so the clamp is protecting the op-amp input.

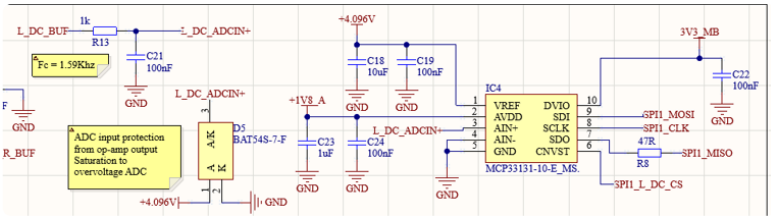
ADC Protection at accidental 36 V

TLDR

- This is added protection, op amp should saturate because of its own input protection

- Saturation can be identified by the ADC measuring a voltage >8V.

BAT54S-7-F | DigiKey Electronics



The rail to rail op amps saturation is beyond the ADCs input range of Vref, thus added BAT54S-7-F | DigiKey Electronics clamped to Vref to protect ADC Inputs

Firmware Equations & Calibration

I have a plan for calibration, based on proven techniques.

DC paths:

- I have 2 points, find the gradient & offset. Year 8 math brutha.

Ripple Paths:

- Get the mean subtracted sine wave of a known sine wave input, AC couple, divide by gain & get a calibration constant.

Calibration: Each measurement path is calibrated in firmware using known external input voltages.

DC Paths: use two-point calibration: V = mCode + b.

- I.e 2 Known points, find gradient & Offset

Term	Meaning
(m)	calibrated gain/slope
(b)	calibrated offset
Code	raw averaged ADC code

Ripple paths: use known AC input amplitude and mean-subtracted waveform capture.
TLDR: Inject a known sine wave, capture the ADC waveform, subtract the mean, convert to volts, divide by the calibrated analogue gain, then report ripple amplitude and frequency & compare.

Example low ripple calibration:

- Input: 100 mVpp sine wave at 1 kHz
- Path: low ripple input
- Measured ADC result after processing: equivalent to 455 mVpp at ADC

The ideal expected gain is:

divder_ratio*G = 0.4545 * 10.09

divder_ratio*G = 4.586

So 100 mVpp at the input should become:

100 mVpp * 4.586 = 458.6 mVpp

Then firmware can convert measured ADC ripple back to node ripple using this calibrated value.

The STM32 will:

1. Capture a block of ADC samples
2. Remove DC bias by averaging samples (Code_mean = average of captured samples)
3. Convert samples into voltage
    a. V_LSB_ADC = 4.096 / 65535 = = 62.5 uV/count
    b. V_AC[i] = Code_AC[i] × 62.5 uV
4. Convert ADC ripple back into input ripple (e.g for low ripple: V_ripple_input = V_ripple_ADC / 4.586)
5. report ripple amplitude and frequency
6. Compare measured result against the known input sine wave and store a correction factor for each ripple path

If the measured value is 455 mVpp, then the calibration factor is:

Cal_ripple,L = 100 mVpp / 455 mVpp

Cal_ripple,L = 0.2198

Spice Simulations